LINEAR ALGEBRA

constants in the system of equations are not important in order to determine if the system is singular or non singular.

Singularity $\Rightarrow$ Rows are linearly dependent.

Determinant

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \Rightarrow$$ Matrix is singular if $[a \;\; b] * k = [c \;\; d]$

$\Rightarrow ak = c$, $bk = d \Rightarrow \frac{c}{a} = \frac{d}{b} = k \Rightarrow ad = bc \Rightarrow ad - bc = 0$

Matrix is singular $\Rightarrow$ Determinant is zero

Matrix row reduction (Gaussian Elimination)

$5a + b = 17$
$4a - 3b = 6$ $\longrightarrow$ $a + 0.2b = 3.4$, $b = 2$ $\longrightarrow$ $a + 0b = 3$, $0a + b = 2$

Diagonal matrix

matrix
Original matrix

$$\begin{pmatrix} 5 & 1 \\ 4 & 3 \end{pmatrix} \longrightarrow \begin{pmatrix} 1 & 0.2 \\ 0 & 1 \end{pmatrix} \longrightarrow \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

Row echelon form

Reduced row echelon form

Row echelon form, the elements on the diagonal are all ones or zeros and everything below the diagonal is zero. upper diagonal

Row operations that preserve singularity

Row operations that p...

1. Switching rows ⇒ $A = \begin{pmatrix} 5 & 1 \\ 4 & 3 \end{pmatrix}$, $|A| = 11$, $B = \begin{pmatrix} 4 & 3 \\ 5 & 1 \end{pmatrix}$, $|B| = -11$

↓ Both non singular.

2. Multiplying a row by a non-zero scalar

3. Adding a row to another

Rank of a matrix

Rank of a matrix measures how much information that matrix or its corresponding system of linear equations is carrying.

System 1

$a+b=0$

$a+2b=0$

Two equations

Two information

Rank = 2

System 2

$a+b=0$

$2a+2b=0$

One piece of information

Rank = 1

System 3

$0a+0b=0$

$0a+0b=0$

No information

Rank = 0

Rank and solution to the system.

$\begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix}$

Dim of solution space (ss) = 0

$\begin{pmatrix} 1 & 1 \\ 2 & 2 \end{pmatrix}$

Dim of ss = 1

$\begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$

Dim of ss = 2

Rank = 2 − Dim. of SS = N(p) − Dim. of SS.

A matrix is non-singular if and only if it has full rank.

Rank is the largest number of linearly independent rows/columns in the matrix.

Rank echelon form

$$\begin{pmatrix} 5 & 1 \\ 4 & -3 \end{pmatrix} \xrightarrow{R_1/5,\ R_2/4} \begin{pmatrix} 1 & 0.2 \\ 1 & -0.75 \end{pmatrix} \xrightarrow{R_2 - R_1} \begin{pmatrix} 1 & 0.2 \\ 0 & -0.95 \end{pmatrix} \xrightarrow{R_2/-0.95} \begin{pmatrix} 1 & 0.2 \\ 0 & 1 \end{pmatrix}$$

$$\begin{pmatrix} 5 & 1 \\ 10 & 2 \end{pmatrix} \xrightarrow{R_1/5,\ R_2/5} \begin{pmatrix} 1 & 0.2 \\ 1 & 0.2 \end{pmatrix} \xrightarrow{R_2 - R_1} \begin{pmatrix} 1 & 0.2 \\ 0 & 0 \end{pmatrix}$$

$\begin{pmatrix} 5 & 1 \\ 4 & -3 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 0.2 \\ 0 & 1 \end{pmatrix}$ → Rank 2 (2 ones in diagonal)

$\begin{pmatrix} 5 & 1 \\ 10 & 2 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 0.2 \\ 0 & 0 \end{pmatrix}$ → Rank 1 (1 one in diagonal)

$\begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$ → Rank 0 (No one in diagonal).

Rank is the number of ones in the diagonal (beauty).

$$\begin{pmatrix} 1 & 1 & 2 \\ 3 & -3 & -1 \\ 2 & -1 & 6 \end{pmatrix} \xrightarrow{R_2 \to R_2/3,\ R_3 \to R_3/2} \begin{pmatrix} 1 & 1 & 2 \\ 1 & -1 & -1/3 \\ 1 & -1/2 & 3 \end{pmatrix}$$

$$\xrightarrow{R_2 \to R_2 - R_1,\ R_3 \to R_3 - R_1} \begin{pmatrix} 1 & 1 & 2 \\ 0 & -2 & -7/3 \\ 0 & 1/2 & 10/3 \end{pmatrix} \xrightarrow{R_2 \to R_2/-2,\ R_3 \to 2R_3} \begin{pmatrix} 1 & 1 & 2 \\ 0 & 1 & 7/6 \\ 0 & 1 & \frac{20}{3} \end{pmatrix}$$

$R_3 \to R_3 - R_2$ $\longrightarrow$ $\begin{pmatrix} 1 & 1 & 2 \\ 0 & 1 & 7/6 \\ 0 & 0 & 11/2 \end{pmatrix}$ $R_3 := \frac{2}{11} R_3$ $\longrightarrow$ $\begin{pmatrix} 1 & 1 & 2 \\ 0 & 1 & 7/6 \\ 0 & 0 & 1 \end{pmatrix}$ Rank = 3.

$\begin{pmatrix} \textcircled{2} & * & * & * & * \\ 0 & \textcircled{1} & * & * & * \\ 0 & 0 & \textcircled{3} & * & * \\ 0 & 0 & 0 & \textcircled{-5} & * \\ 0 & 0 & 0 & 0 & \textcircled{1} \end{pmatrix}$ Rank = 5

$\begin{pmatrix} \textcircled{3} & * & * & * & * \\ 0 & 0 & \textcircled{1} & * & * \\ 0 & 0 & 0 & \textcircled{-4} & * \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$ Rank = 3.

→ If the matrix has zero rows, they need to be at the bottom.

→ Every row has a pivot (left most non-zero entry) [pivots are 0 in matrix above

→ Every pivot is to the right of the pivots on the row above.

→ Rank of the matrix is the number of pivots.

Reduced row echelon form

Original matrix $\begin{pmatrix} 5 & 1 \\ 4 & -3 \end{pmatrix}$ $\longrightarrow$ Intermediate matrix $\begin{pmatrix} 1 & 0.2 \\ 0 & 1 \end{pmatrix}$ Row echelon form $\longrightarrow$ diagonal matrix $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ Reduced row echelon form

→ Is in row echelon form

→ Each pivot is a 1

→ Any number above a pivot is a

→ Rank of matrix is the number of pivots.

$\begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \end{pmatrix}$ $\underset{R_2 \to R_2 - 4R_3}{\overset{R_1 \to R_1 - 2R_2}{\longleftrightarrow}}$ $\begin{pmatrix} 1 & 0 & -5 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$ $R_1 \to R_1 + 5R_3$ $\longrightarrow$ $\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

$\begin{pmatrix} 0 & 0 & 1 \end{pmatrix}$

Gaussian Elimination Method

$$\begin{aligned} 2a - b + c &= 1 \\ 2a + 2b + 4c &= -2 \\ 4a + b + 0c &= -1 \end{aligned}$$

Augmented matrix $= \left(\begin{array}{ccc|c} 2 & -1 & 1 & 1 \\ 2 & 2 & 4 & -2 \\ 4 & 1 & 0 & -1 \end{array}\right)$

$R_1 \leftarrow R_1/2 \longrightarrow \left(\begin{array}{ccc|c} 1 & -1/2 & 1/2 & 1/2 \\ 2 & 2 & 4 & -2 \\ 4 & 1 & 0 & -1 \end{array}\right) R_2 \leftarrow R_2 - 2R_1 \longrightarrow \left(\begin{array}{ccc|c} 1 & -1/2 & 1/2 & 1/2 \\ 0 & 3 & 3 & -3 \\ 4 & 1 & 0 & -1 \end{array}\right)$

$R_2 \leftarrow R_2/3 \left(\begin{array}{ccc|c} 1 & -1/2 & 1/2 & 1/2 \\ 0 & 1 & 1 & -1 \\ 4 & 1 & 0 & -1 \end{array}\right) R_3 \leftarrow R_3 - 4R_1 \longrightarrow \left(\begin{array}{ccc|c} 1 & -1/2 & 1/2 & 1/2 \\ 0 & 1 & 1 & -1 \\ 0 & 3 & -2 & -3 \end{array}\right)$

$R_3 \leftarrow R_3 - 3R_2 \left(\begin{array}{ccc|c} 1 & -1/2 & 1/2 & 1/2 \\ 0 & 1 & 1 & -1 \\ 0 & 0 & -5 & 0 \end{array}\right) R_3 \leftarrow R_3/-5 \longrightarrow \left(\begin{array}{ccc|c} 1 & -1/2 & 1/2 & 1/2 \\ 0 & 1 & 1 & -1 \\ 0 & 0 & 1 & 0 \end{array}\right)$

$R_1 \leftarrow 2R_1 + R_2 \left(\begin{array}{ccc|c} 1 & 0 & 3 & 0 \\ 0 & 1 & 1 & -1 \\ 0 & 0 & 1 & 0 \end{array}\right) R_1 \leftarrow R_1 - 3R_3 \longleftarrow \left(\begin{array}{ccc|c} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & -1 \\ 0 & 0 & 1 & 0 \end{array}\right)$

$R_2 \leftarrow R_2 - R_3 \left(\begin{array}{ccc|c} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 0 \end{array}\right)$

$a = 0, b = -1, c = 0$

$\left(\begin{array}{ccc} 1 & 3 & 2 \\ 2 & 1 & 5 \end{array}\right) R_2 \leftarrow 2R_1 - R_2, R_3 \leftarrow 3R_1 - R_3 \longleftarrow \left(\begin{array}{ccc} 1 & 3 & 2 \\ 0 & 5 & -1 \\ & 5 & 2 \end{array}\right) R_3 \leftarrow R_3 - R_2 \longleftarrow \left(\begin{array}{ccc} 1 & 3 & 2 \\ 0 & 5 & -1 \\ 0 & 0 & -3 \end{array}\right)$

$$\begin{pmatrix} 3 & 4 & 4 \end{pmatrix} \qquad \begin{pmatrix} 0 & \cdot & \cdot \end{pmatrix}$$

Gaussian Elimination Algorithm

The aim is to put the matrix in row echelon form and then use back substitution to solve for each variable.

$$M = \begin{pmatrix} * & * & * \\ 0 & \text{pivot} & * \\ 0 & \text{value} & * \end{pmatrix} \begin{matrix} \rightarrow R_0 \\ \rightarrow R_1 \\ \rightarrow R_2 \end{matrix}$$

To nullify last row (Row 2), two steps are required:

1. Scale R_1 by the inverse of the pivot: $R_1 \rightarrow \frac{1}{\text{pivot}} \cdot R_1$

Resulting in the updated matrix with the pivot for row 1 set to 1:

$$M = \begin{pmatrix} * & * & * \\ 0 & 1 & * \\ 0 & \text{value} & * \end{pmatrix}$$

2. Next, to eliminate the value below the pivot in row 1, apply:

$R_2 \rightarrow R_2 - \text{value} \cdot R_1$ (Can we just make it $R_2 \rightarrow R_2 - \frac{\text{value}}{\text{pivot}}^{?} R_1$)

The transformation yields the modified matrix:

$$M = \begin{pmatrix} * & * & * \\ 0 & 1 & * \\ 0 & 0 & * \end{pmatrix}$$

Consider this example:

$$\begin{aligned} 2x_2 + x_3 &= 3 \\ x_1 + x_2 + x_3 &= 6 \\ & \end{aligned} \qquad A = \begin{pmatrix} 0 & 2 & 1 \\ 1 & 1 & 1 \\ & 2 & 1 \end{pmatrix}, B = \begin{pmatrix} 3 \\ 6 \\ 12 \end{pmatrix}$$

$x_1 + 2x_2 + x_3 - 12$ (

We need to first augment the matrix: $M = \left(\begin{array}{ccc|c} 0 & 2 & 1 & 3 \\ 1 & 1 & 1 & 6 \\ 1 & 2 & 1 & 12 \end{array}\right)$

Step 1: Starting with row 0: the initial candidate for the pivot is always the value in the main diagonal of the matrix, in this case M[0,0] , row 0 $\rightarrow R_0$

$R_0 = (0 \ 2 \ 1 \mid 3)$

Since the initial candidate here is 0, we need to use the function 'get_index_first_non_zero_value_from_column' (M, 0, 0)

this returns 1, then we do a 'Swap_rows' (M, 0, 1)

We now have $M = \left(\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 2 & 1 & 3 \\ 1 & 2 & 1 & 12 \end{array}\right)$

Since the pivot is already 1, we don't need to do row scaling.

$\Rightarrow R_1 \rightarrow R_1 - 0 \cdot R_0 = R_1$

Moving to third row (R_2) the value in the augmented matrix below the pivot from R_0 is M[2,0] which is 1

$R_2 \Rightarrow R_2 - 1 \cdot R_0 = (0 \ 1 \ 0 \mid 6)$

$\Rightarrow M = \left(\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 2 & 1 & 3 \\ 0 & 1 & 0 & 6 \end{array}\right)$ $\longrightarrow$ All operations for R_0 is done

Progressing to second row (R_1), the value in the main diagonal is 2, different from 0. Scaling it by 1/2

$\Rightarrow M = \left(\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 1 & 1/2 & 3/2 \\ 0 & 1 & 0 & 6 \end{array}\right)$

Now there is only one row below it for row replacement. The value just below the pivot is located at $M[2,1]$ which is 1. Thus:

$$R_2 \to R_2 - 1 \cdot R_1 = (0 \;\; 0 \;\; -1/2 \,|\, 9/2)$$

$$\Rightarrow M = \left(\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 1 & 1/2 & 3/2 \\ 0 & 0 & -1/2 & 9/2 \end{array}\right)$$

Finally, normalizing last row as: $R_2 \to -2 \cdot R_2$

$$M = \left(\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 1 & 1/2 & 3/2 \\ 0 & 0 & 1 & -9 \end{array}\right)$$

And we are done with row echelon form

Back substitution

Aim is to end up with a reduced echelon form

Formula is: $R_{\text{row above}} \to R_{\text{row above}} - \text{value} \cdot R_{\text{row pivot}}$

$$R_i \to R_i - \text{value} \cdot R_{i+1 \text{ pivot}}$$

$$M = \left(\begin{array}{ccc|c} 1 & -1 & 1/2 & 1/2 \\ 0 & 1 & 1 & -1 \\ 0 & 0 & 1 & -1 \end{array}\right)$$

Starting from bottom to top:

* R_2:
  - $R_1 = R_1 - 1 \cdot R_2 = (0 \;\; 1 \;\; 0 \,|\, 0)$
  - $R_0 = R_0 - \frac{1}{2} R_2 = (1 \;\; -1 \;\; 0 \,|\, 1)$

$$\Rightarrow M = \left(\begin{array}{ccc|c} 1 & -1 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -1 \end{array}\right)$$

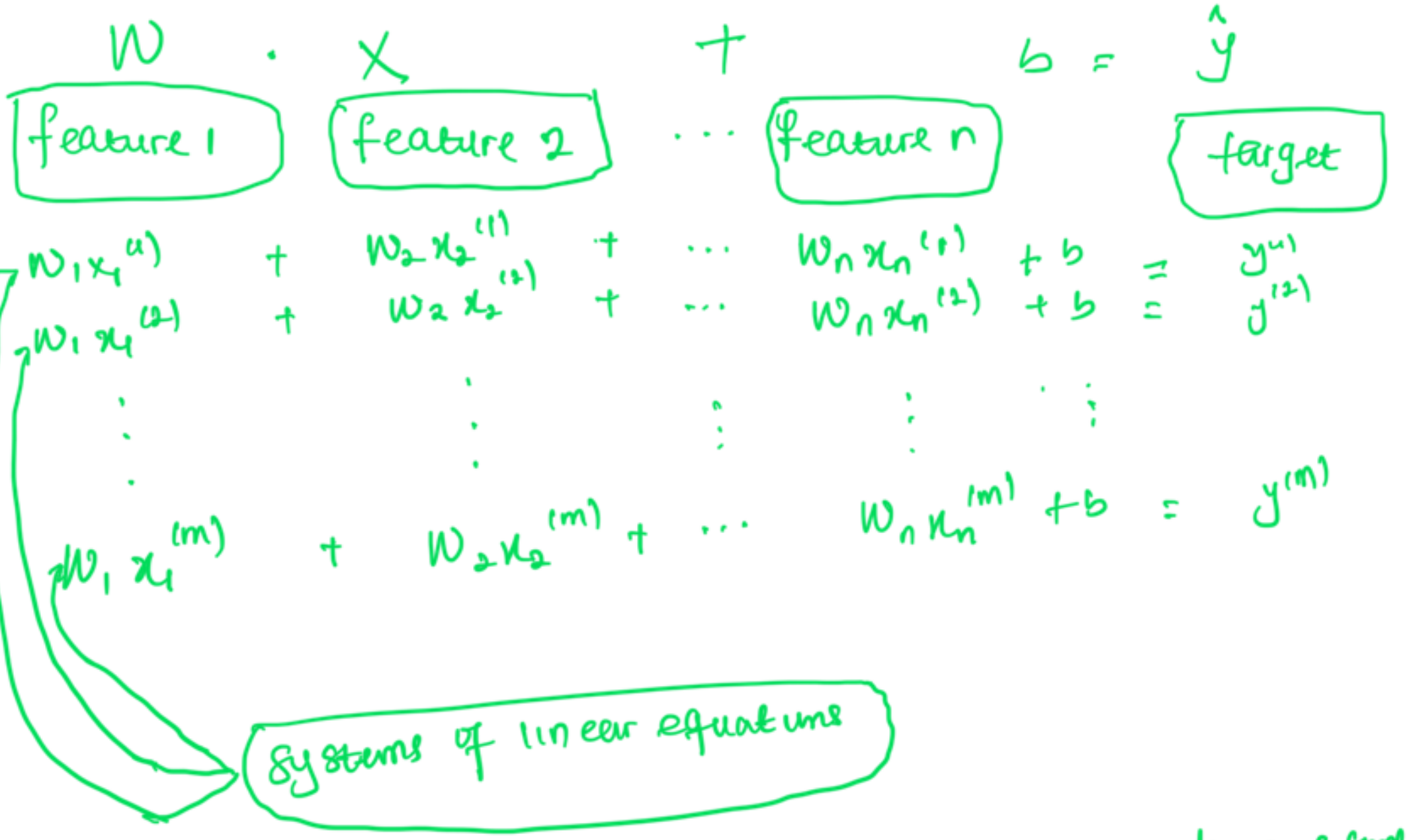
* R_1:
  - $R_0 = R_0 - (-1 \cdot R_1) = (1 \;\; 0 \;\; 0 \,|\, 1)$

$$\Rightarrow M = \left(\begin{array}{ccc|c} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -1 \end{array}\right)$$

Vectors and Linear Transformations

Machine learning motivation

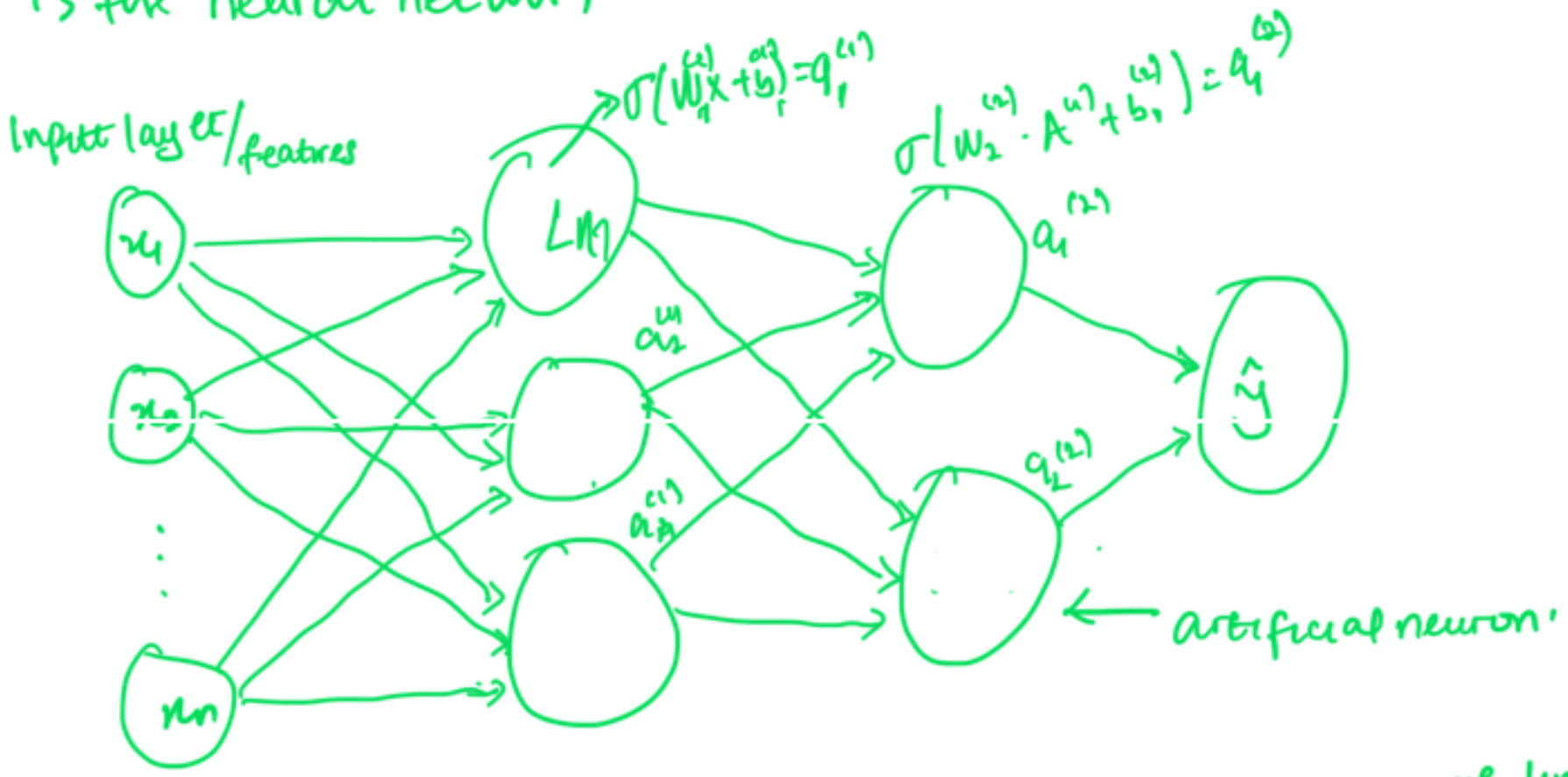
Linear Regression is an example of a machine learning model where you treat your datasets as a system of linear equations.



The way it works is that you have some data sets made up of some features called $x_1, x_2, \ldots, x_n$ and then you have multiple examples in your data sets (multiple rows of x's in the feature matrix) and superscript along each row denoting the number of each examples and vector of targets or y values. The idea of linear regression is that you come up with a set of weights $w_1, w_2, \ldots, w_n$ one for each set of feature as well as a bias value that allows you to characterize this data set as a system of linear equations.

$$W \cdot X + b = \hat{y}$$

Real world datasets are not typically the systems of equations that you can solve analytically. But the assumption that this dataset could be approximated as a systems of linear equations turns out to be a reasonable one. Then machine learning will allow us to solve this in an iterative fashion and make predictions on the target $\hat{y}$ for any new set of x values. But we can only get so far by approximating real world datasets with linear models, because in many cases the relationship between features and targets is nonlinear. One of the most powerful machine learning models for representing nonlinear systems is the neural network



Neural networks under the hood are just a large collection of linear models.

Vectors and their properties

$x = (x_1, x_2, \ldots, x_n)$

L1 norm: $\|x\|_1 = |x_1| + |x_2| + \cdots + |x_n|$

L2 norm: $\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$

$x = (x_1, x_2, \ldots, x_n)$, $y = (y_1, y_2, \ldots, y_n)$

$x + y = (x_1 + y_1, x_2 + y_2, \ldots x_n + y_n)$

$x - y = (x_1 - y_1, x_2 + y_2, \ldots, x_n + y_n)$

$$\lambda x = (\lambda x_1, \lambda x_2, \dots, \lambda x_n)$$

$$\langle x, y \rangle = x \cdot y = x_1 y_1 + x_2 y_2 + \dots + x_n y_n$$

$$\text{L2 norm} = \sqrt{\text{dot product}(u,u)} = \sqrt{\langle u, u \rangle} = \|u\|_2$$

$$\langle x, y \rangle = x \cdot y^T = (x_1, x_2, \dots, x_n) \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}$$

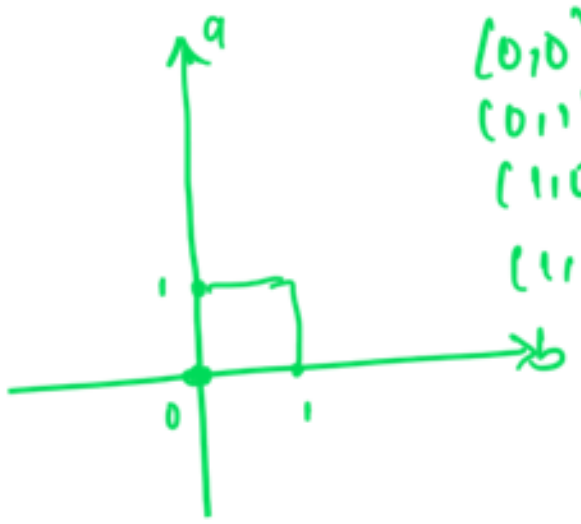
If two vectors are orthogonal, $\langle u, v \rangle =$

$$\langle u, u \rangle = |u|^2 = |u| \cdot |u|$$

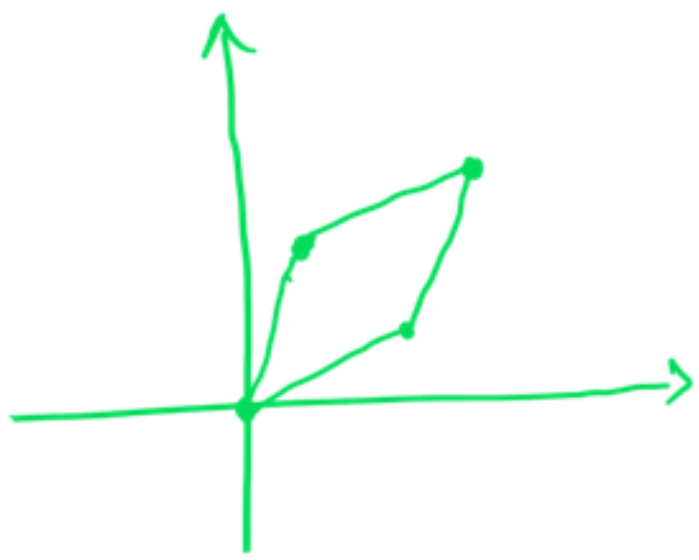
$$\langle u, v \rangle = |u| \cdot |v| = |u||v| \cos\theta \Rightarrow \cos\theta = \frac{u \cdot v}{|u||v|}$$

Matrices as linear transformations

$$A = \begin{pmatrix} 3 & 1 \\ 1 & 2 \end{pmatrix} \quad \begin{matrix} a & b \end{matrix}$$



$(0,0) \to (0,0)$
$(0,1) \to (3,1)$
$(1,0) \to (1,2)$
$(1,1) \to (4,3)$



Linear transformations as matrices

$(0,0) \to (0,0)$
$(1,0) \to (3,-1)$
$(0,1) \to (2,3)$
$(1,1) \to (5,2)$

then the matrix is $\begin{pmatrix} 3 & -1 \\ 2 & 3 \end{pmatrix}$

→ Matrix multiplication

take rows from first matrix and column from second matrix and take dot product for each entries ...

→ Identity matrix

→ Matrix inverse

Neural networks and matrices

$T(v) = av$, $T(x,y) = (ax, ay)$ where $v = (x,y)$

$e_1 = (1,0)$ and $e_2 = (0,1)$

$T(e_1) = T((1,0)) = (a,0)$ and $T(e_2) = T((0,1)) = (0,a)$

$$M = \begin{pmatrix} a & 0 \\ 0 & a \end{pmatrix}$$

$T(x,y) = (x+my, y)$

$T(e_1) = (1,0)$, $T(e_2) = (m,1) \Rightarrow M = \begin{pmatrix} 1 & m \\ 0 & 1 \end{pmatrix}$

$$M = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

$$T(e_1) = e_1 \cdot M = \begin{pmatrix} 1 \\ 0 \end{pmatrix}\begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} = \begin{pmatrix} \cos\theta \\ \sin\theta \end{pmatrix}$$

$$T(e_2) = e_2 \cdot M = \begin{pmatrix} 0 \\ 1 \end{pmatrix}\begin{pmatrix} \cos\theta & \sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} = \begin{pmatrix} -\sin\theta \\ \cos\theta \end{pmatrix}$$

$T\begin{pmatrix} v_1 \\ v_2 \end{pmatrix}$ $e_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ $e_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$ $[e_1\ e_2] = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I$

$A[e_1\ e_2] = AI = A$

$L(e_1) = Ae_1$ and $L(e_2) = Ae_2$

$[L(e_1)\ L(e_2)]$

$$A[e_1\ e_2] = [Ae_1\ Ae_2]$$

$$T((x,y)) = (ax, ay)$$

$$T(e_1) = (a,0),\ T(e_2) = (0,a)$$

$$A = [T(e_1)\ T(e_2)] = \begin{bmatrix} a & 0 \\ 0 & a \end{bmatrix}$$

$$T((x,y)) = (x+my, y)$$

$$T(e_1) = (1,0),\ T(e_2) = (m,1)$$

$$A = \begin{pmatrix} 1 & m \\ 0 & 1 \end{pmatrix}$$

$$R = \begin{pmatrix} e_1 & e_2 \\ \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

$$T(e_1) = T((1,0)) = (\cos\theta, \sin\theta)$$

$$T(e_2) = T((0,1)) = (-\sin\theta, \cos\theta)$$

$$T((x,y)) = (x\cos\theta - y\sin\theta,\ x\sin\theta + y\cos\theta)$$

r → rotation

s → stretch

$$T_{rands}(v) = (T_r \circ T_s)(v) = T_r(T_s(v))$$

$$\begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}\begin{pmatrix} a & 0 \\ 0 & a \end{pmatrix} = \begin{pmatrix} a\cos\theta & -a\sin\theta \\ a\sin\theta & a\cos\theta \end{pmatrix}$$

→ Implement a neural network with a single perceptron and two input nod

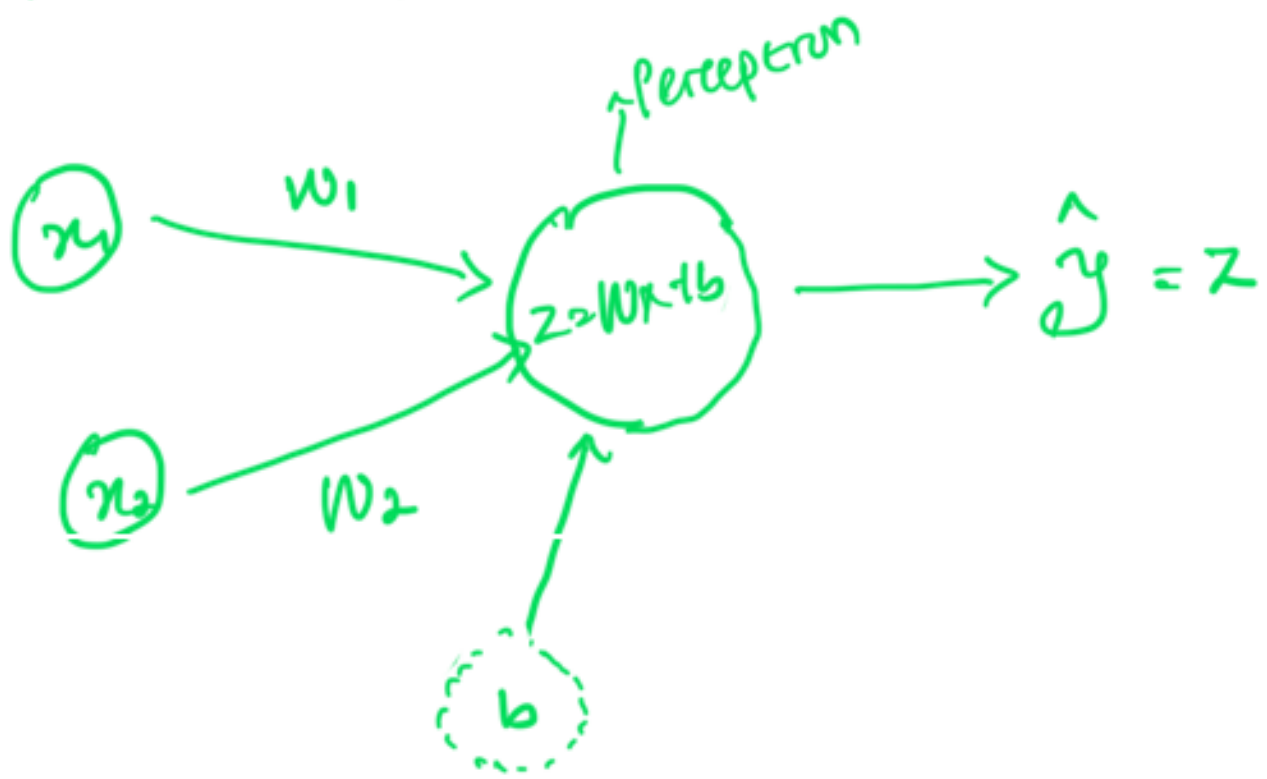
for linear regression.

→ Implement forward propagation using matrix multiplication.

$$\hat{y} = w_1 x_1 + w_2 x_2 + b = Wx + b$$

where Wx is the dot product of the input vectors $x = [x_1 \; x_2]$ and parameters vector $W = [w_1, w_2]$, scalar parameter b is the intercept

The goal is to find the best parameters w_1, w_2 and b such that the difference between original values y_i and predicted values $\hat{y}_i$ are minimum. Neural network does that and matrix multiplication is in the core of the model.



The perceptron output calculation for a training example $x = [x_1 \; x_2]$ can be written as the dot product:

$$z = w_1 x_1 + w_2 x_2 + b = Wx + b$$

where weights are in the vector $W = [w_1, w_2]$ and bias b is a scalar.

The output layer will have a single node $\hat{y} = z$.

$$Wx = [w_1 \; w_2] \begin{bmatrix} x_1^{(1)} & x_1^{(2)} & \cdots & x_1^{(m)} \\ x_2^{(1)} & x_2^{(2)} & \cdots & x_2^{(m)} \end{bmatrix}$$

$$= [w_1 x_1^{(1)} + w_2 x_2^{(1)} \; + w_1 x_1^{(2)} + w_2 x_2^{(2)} + \cdots + w_1 x_1^{(m)} + w_2 x_2^{(m)}]$$

model can be written as

and the model can be

$$Z = W \times + b$$

$$\hat{Y} = Z$$

Where b is broadcasted to the vector of a size 1xm). These are the calculations to perform in the forward propagation step.

Now we can compare the resulting vector of the prediction $\hat{Y}(1 \times m)$ with the original vector of data Y. This can be done with the so called cost function that measures how close the vector of predictions is to the training data. For our simple neural network, we can use:

$$L(w,b) = \frac{1}{2m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)}\right)^2$$

The aim is to minimize the cost function during training, which will minimize the difference between original values and predicted values.

When weights were just initialized with some random values and NO training was done yet, we can't expect good results.

The next step is to adjust the weights and bias, in order to minimize the cost function. This process is called backward propagation and is done iteratively: we update the parameters with a small change and repeat the process.

General methodology to build neural networks is to:

→ Define the neural network structure (# of inputs, # of hidden units, etc)

→ Initialize the model's parameters.

→ Loop:

→ Implement forward propagation (calculate perceptron output)

→ Implement backward propagation (to get required correction for the parameters)

→ update parameters

→ Make predictions.

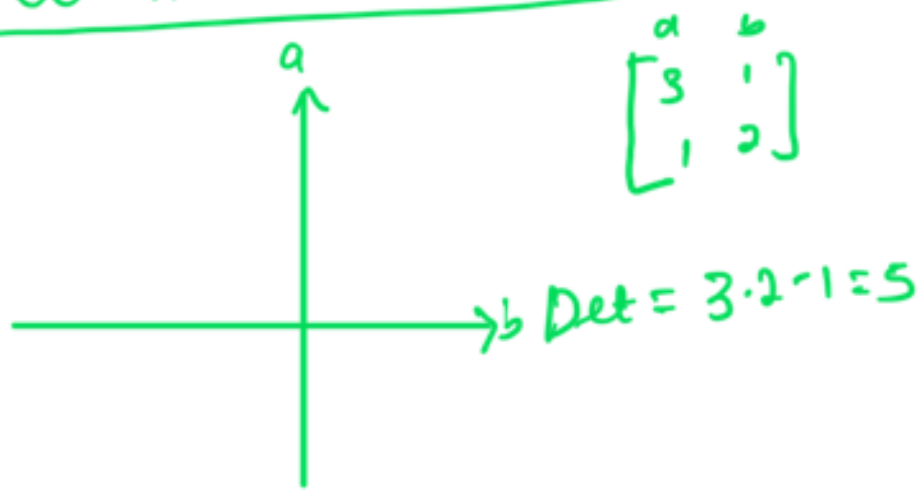
Determinants and Eigenvectors

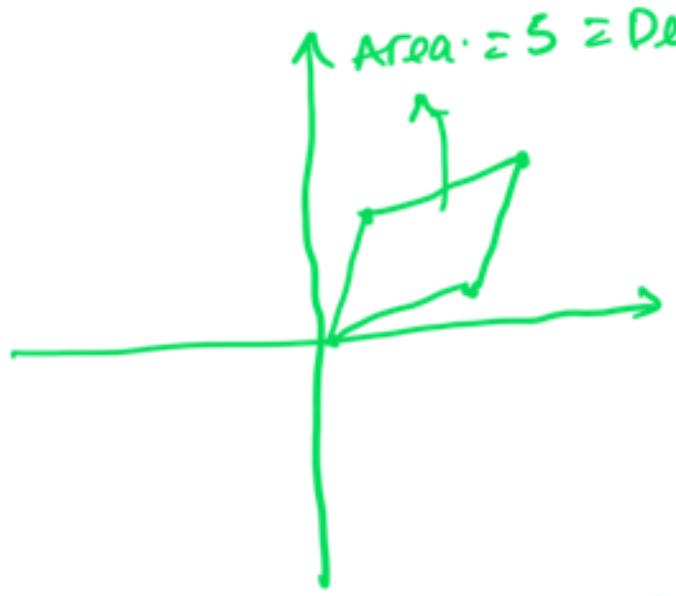
Singularity and rank of linear transformations

If the resulting point after multiplying it by a matrix covers the entire plane then the transformation is non singular and vice versa.

The rank is also the dimension of the image of the transformation

Determinant as an area





The determinant of the matrix is the area of the image of the fundamental basis formed by the unit square

for singular transformations, this also is the same as determinant which is zero area.

Determinant as a product

$|AA^{-1}| = |I| = 1$

$|A^{-1}| = \frac{1}{|A|}$

$\det(AB) = \det(A)\det(B) \Rightarrow |AB| = |A||B|$

$$|A^{-1}| = \frac{1}{|A|}$$

⟶ Bases

Every point in a space can be expressed as a linear combination of

elements in the basis.

two vectors in the same direction cannot form a basis.

⟶ Span

The span of a set of vectors is simply the set of points can be reached by walking in the direction of these vectors in any combinatio

A basis is a minimal spanning set

Number of elements in the basis is the dimension.

A group of vectors is said to be linearly independent if none of the vectors in the group can be obtained as a linear combination of th others otherwise they are linearly dependent

A basis is a set of vectors that satisfies two conditions:

→ Spans a vector space.

→ is linearly independent.

⟶ Eigenbases

A basis that stretches the transformation

the vectors in the spaces are called eigenvectors and the ~~Stretch~~ stretching value is called eigen values.

⟶ Eigenvectors and Eigenvalues. ($AV_1 = \lambda_1 V_1$, $AV_2 = \lambda_2 V_2$)

turns matrix multiplication into scalar multiplication (very important for machine learning which does millions of matrix multiplication)

also since you have an eigen basis, you can express every other element in the vector space as a linear combination of the bases

we use the eigenvalues and vectors to turn matrix multiplicatio

and the rise the ~~eigenvalues~~ into scalar multiplication.

For every point in the plane, you need to calculate the inverse of the eigenbasis and the multiply by your vector so this is still alot of work and still involves matrix multiplication. So its not that eigen vectors remove all the work but they let you decide when you want to do the work. In some machine learning contexts, it may be worth to calculate these coordinates ahead of time so that when its time to apply a transformation, you can do it much more faster.

* $Av = \lambda v$ for each eigenvector/eigenvalue
* Eigenvectors: direction of stretch
* Eigenvalues: how much they stretch
* Eigenbasis: the set of a matrix eigenvectors, can be arranged with one eigenvector in each column.

* Save work and characterize linear transformations.

$A = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$, $Av = \lambda v \Rightarrow Av - \lambda v = 0 = (A - \lambda I)v = 0$

$|A - \lambda I||v| = 0 \Rightarrow A - \lambda I = 0$

$\begin{vmatrix} 2-\lambda & 1 \\ 0 & 3-\lambda \end{vmatrix} = (2-\lambda)(3-\lambda) = 0 \Rightarrow \lambda = 2 \text{ or } \lambda = 3.$

for $\lambda = 2$

$\begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}\begin{pmatrix} x \\ y \end{pmatrix} = 2\begin{pmatrix} x \\ y \end{pmatrix} \Rightarrow \begin{matrix} 2x+y = 2x \\ 0x+3y = 2y \end{matrix} \Rightarrow \begin{matrix} x = 1 \\ y = 0 \end{matrix} \quad \begin{pmatrix} 1 \\ 0 \end{pmatrix}$

for $\lambda = 3$

$\begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}\begin{pmatrix} x \\ y \end{pmatrix} = 3\begin{pmatrix} x \\ y \end{pmatrix} \Rightarrow \begin{matrix} 2x+y = 3x \\ 0x+3y = 3y \end{matrix} \Rightarrow \begin{matrix} x = 1 \\ y = 1 \end{matrix} \quad \begin{pmatrix} 1 \\ 1 \end{pmatrix}$

→ Dimensionality Reduction

* Reduce dimensions (# of columns) of dataset.
* Preserve as much as information as possible.

→ Projections.

To project matrix A into a vector v, we do.

$$A_p^{2} \quad A \quad \frac{v}{\|v\|_2} \quad \Rightarrow \quad A_p = Av$$

A_p: rx1, A: rxc, $\frac{v}{\|v\|_2}$: cx1

→ Principal Component Analysis.

for PCA, more spread out data points preserve more information from the original dataset. Preserving more spread preserves more information

Goal of PCA is to find the projection that preserves the maximum possible in the data.

→ Benefits of Dimensionality Reduction

- ▷ Easier dataset to manage
- ▷ PCA reduces dimensions while minimizing information loss.
- ▷ Simpler visualization

→ Variance and Covariance

$$\text{Mean}(x) = \frac{1}{n}\sum_{i=1}^{n} x_i \quad \rangle \text{ average}$$

$$\text{Mean}(y) = \frac{1}{n}\sum_{i=1}^{n} y_i$$

$$\text{Variance} = \frac{\sum_{i=1}^{n} |x_i - \text{Mean}(x)|^2}{n}$$

variance determines how spread out

$$Var(x) = Variance(x) = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \text{Mean}(x))^2$$ the data is:

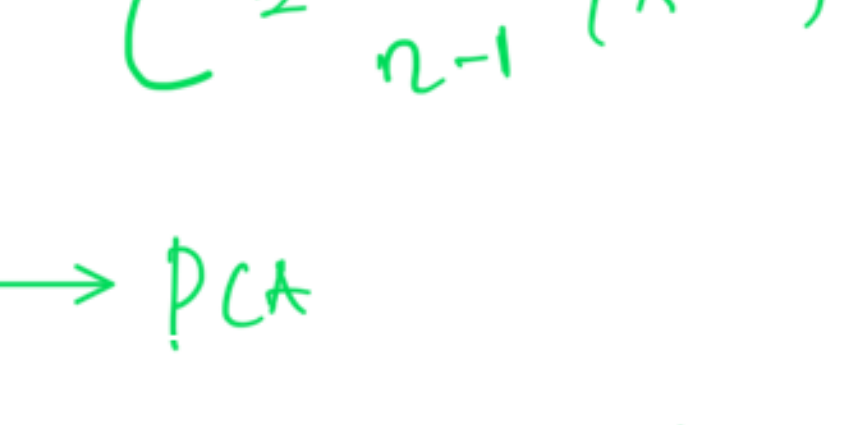
$$Var(y) \; Variance(y) = \frac{1}{n-1}\sum_{i=1}^{n}(y_i - \text{Mean}(y))^2$$

Covariance measures how two features of a dataset varies with respect to one another.

$$Cov(x,y) = \frac{1}{n-1}\sum_{i=1}^{n}(x-\mu_x)(y-\mu_y)$$ direction of the relationship between two variables.

$$\Rightarrow Cov(x,x) = Var(x)$$

$\longrightarrow$ Covariance matrix.



$$C = \begin{matrix} & x & y \\ x & Var(x) & Cov(x,y) \\ y & Cov(y,x) & Var(y) \end{matrix}$$

$$= \begin{bmatrix} Cov(x,x) & Cov(x,y) \\ Cov(y,x) & Cov(y,y) \end{bmatrix}$$

$$A = \begin{bmatrix} x_1 & y_1 \\ x_2 & y_2 \\ \vdots & \vdots \\ x_n & y_n \end{bmatrix} \quad \mu = \begin{bmatrix} \mu_x & \mu_y \\ \mu_x & \mu_y \\ \vdots & \vdots \\ \mu_x & \mu_y \end{bmatrix}$$

$$C = \frac{1}{n-1}(A-\mu)^T(A-\mu)$$

$\longrightarrow$ PCA

$\longrightarrow$ Data $\longrightarrow$ Covariance Matrix $\longrightarrow$ Eigen value/vectors (Principal components)

$\longrightarrow$ find the largest eigen value (more variance) $\longrightarrow$ Project dat on the line

→ PCA Mathematical formulation

You have n observations of m variables $(x_1, x_2, \dots, x_m)$

Goal: Reduce to 2 variables

1. Create matrix.

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1m} \\ x_{21} & x_{22} & \cdots & x_{2m} \\ \vdots & \vdots & \ddots & \\ x_{n1} & x_{n2} & \cdots & x_{nm} \end{bmatrix}$$

m vars; n observations

2. Center the data

$$X - \mu = \begin{bmatrix} x_{11} - \mu_1 & x_{12} - \mu_2 & \cdots & x_{1m} - \mu_m \\ x_{21} - \mu_1 & x_{22} - \mu_2 & \cdots & x_{2m} - \mu_m \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} - \mu_1 & x_{n2} - \mu_2 & \cdots & x_{nm} - \mu_m \end{bmatrix}$$

3. Calculate Covariance matrix

$$C = \frac{1}{n-1}(X-\mu)^T(X-\mu) = \begin{bmatrix} \mathrm{Var}(X_1) & \mathrm{Cov}(X_1, X_2) & \cdots & \mathrm{Cov}(X_1, X_m) \\ \mathrm{Cov}(X_1, X_2) & \mathrm{Var}(X_2) & \cdots & \mathrm{Cov}(X_2, X_m) \\ \vdots & \vdots & \ddots & \vdots \\ \mathrm{Cov}(X_1, X_m) & \mathrm{Cov}(X_2, X_m) & \cdots & \mathrm{Var}(X_m) \end{bmatrix}$$

4. Calculate Eigenvalues and Eigenvectors. and order them in order of the largest eigenvalue

Big ↑ Small

$\lambda_1 \quad v_1$

$\lambda_2 \quad v_2$

$\vdots \quad \vdots$

$\lambda_m \quad v_m$

5. Create projection matrix, since our goal is 2 variables, we only keep the first two.

$$V = \begin{bmatrix} \frac{v_1}{\|v_1\|_2} & \frac{v_2}{\|v_2\|_2} \end{bmatrix}$$

6. Project centered Data

$$X_{PCA} = (X - \mu) V$$

→ Discrete dynamical systems

if today is

Tomorrow will be	Sunny	Cloudy	Rainy
Sunny	0.80	0.45	0.30
Cloudy	0.15	0.35	0.40
Rainy	0.05	0.20	0.30

State vector → today is cloudy.

X_0

0
1
0

=

Tomorrow will be

X_1

0.45
0.35
0.2

Tomorrow dot product

1 day ahead prediction dot product.

All values are positive and columns add up to 1 ⇒ markov matrix

Transition matrix

P

0.8	0.45	0.3
0.15	0.35	0.4
0.05	0.2	0.3

Equilibrium vector

X_∞

0.6665
0.2223
0.1112

→ X_∞ is an eigen vector of P

$$M = \begin{pmatrix} 3 & 2 \\ 5 & 8 \end{pmatrix}$$

$$n - M = \begin{pmatrix} -1 & -3 \\ 1 & 3 \end{pmatrix}$$

$$(n-M)(n-M)^T = \begin{pmatrix} -1 & -3 \\ 1 & 3 \end{pmatrix}\begin{pmatrix} -1 & 1 \\ -3 & 3 \end{pmatrix} = \begin{pmatrix} 10 & -10 \\ -10 & 10 \end{pmatrix}$$

Discrete Dynamical Systems

A discrete dynamical system describes a system where as time goes by, the state changes according to some process. When defining these systems, we could represent all the possible states in a vector called the state vector.

Each DDS can be represented by a transition matrix P (markov matrix) which indicates, given a particular state, what are the chances or probabilities of moving to each of the other states.

Starting with an initial state X_0, the transition to the next state X_1 is a linear transformation defined by the transition matrix $P: X_1 = PX_0$. That leads to $X_2 = PX_1 = PX_0^2 \dots$

$$\Rightarrow X_t = PX_{t-1} = PX_0^t \text{ for } t = 1, 2, \dots$$

Finding eigenvalues and eigenvectors of the transition matrix P will always give the eigenvalue which will be one and the rest will be less than one - this is a property of transition matrix and they have so many other properties. Transition matrices are Markov matrix.

A square matrix whose entries are all non negative and the sum of elements for each column is 1 is called a markov matrix. They have a handy property, they always have an eigenvalue equal to 1.

Point is: $X_m = PX_{m-1}$

as $m \to \infty$, $X_m \approx X_{m-1} \approx \pi \Rightarrow \pi = P\pi$ | $P\pi = \pi$ 1 eigenvalue

$\Rightarrow P\pi = 1\cdot\pi$

π is the eigenvector corresponding

$X_m = P X_{m-1} = 1 \times X_{m-1}$

This means predicting probabilities at time m when m is large enough, you can simply just look for an eigenvector corresponding to eigenvalue 1.

M is 55×4096,

Get M vector, the shape of that would be 4096×1

So we have $M = [M_1, M_2, \ldots, M_{4096}]$

repeat 55 times

$M = [M_1, M_1 \ldots M_1, M_2 \ldots M_2 \ldots M_{4096} \ldots M_{4096}]$ 55×4096

So now, we reshape

$$\begin{bmatrix} u_1 & u_2 & \cdots & u_{4096} \\ u_1 & u_2 & \cdots & u_{4096} \\ \vdots & \vdots & & \vdots \\ u_1 & u_2 & \cdots & u_{4096} \end{bmatrix}$$

$$\text{Cov matrix} = \frac{1}{n-1}(A-\mu)^T(A-\mu)$$

→ Explained Variance

When deciding how many components to use for the dimensionality reduction, one good criteria to consider is the explained variance. The explained variance is measure of how much variation in a dataset can be

attributed to each of the principal components (eigenvectors). In words, it tells us how much of the total variance is "explained" by each component.

In PCA, the first principal component i.e the eigenvector associated to the largest eigenvalue is the one with the greatest explained variance.

As the goal of PCA is to reduce dimensionality by projecting data in the directions with biggest variability.

Explained variance of a PC v_i = $\frac{\lambda_i}{\sum_{i=1}^{55} \lambda_i}$